MANCHESTER 1824
The University of Manchester

A Modular Real-Time Haptic Rendering System Integrating Physics-Based Simulation and OpenGL Visualisation

A dissertation submitted to The University of Manchester for the degree of

Bachelor of Engineering in Mechatronic Engineering

in the Faculty of Science and Engineering

Year of submission

2026

Student ID

11349510

School of Engineering

Abstract

This is abstract text.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Declaration of originality

I hereby confirm that no portion of the work referred to in the thesis has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright statement

i The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the "Copyright") and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.

ii Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made *only* in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.

iii The ownership of certain Copyright, patents, designs, trademarks and other intellectual property (the "Intellectual Property") and any reproductions of copyright works in the thesis, for example graphs and tables ("Reproductions"), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.

iv Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see `http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420`), in any relevant Thesis restriction declarations deposited in the University Library, The University Library's regulations (see `http://www.library.manchester.ac.uk/about/regulations/`) and in The University's policy on Presentation of Theses.

Acknowledgments

Acknowledgments go here.

1 Introduction

1.1 Background and Motivation

Robotic manipulators are increasingly deployed in domains such as industrial automation, teleoperation, and robot-assisted surgery. In many of these applications, operator training is essential to ensure safe and efficient task execution. While visual feedback provides global situational awareness, it is often insufficient for developing fine motor skills, accurate force regulation, and intuitive understanding of interaction constraints.

Haptic interfaces enable bidirectional physical interaction between a human operator and a robotic or simulated environment. By rendering interaction forces to the user, such systems can improve perception of contact conditions, enhance motor learning, and support safer manipulation strategies during training tasks. As robotic systems become more widely adopted in both research and educational settings, there is increasing motivation to develop accessible haptic training platforms that allow users to experience realistic interaction dynamics without requiring direct access to expensive physical manipulators.

Despite these motivations, many existing haptic training solutions remain costly, proprietary, and tightly coupled to specific software ecosystems. This limits transparency of the control architecture and restricts the ability of researchers and students to experiment with alternative control strategies, custom simulation environments, or novel hardware configurations. There is therefore a practical need for modular and low-cost haptic interface systems that enable real-time bidirectional interaction with robotic manipulation simulations while remaining flexible and extensible.

1.2 Research Gap

While prior work has addressed either high-fidelity haptic rendering or low-cost robotic hardware in isolation, there is limited work that integrates low-cost actuated haptic devices with fully transparent, simulator-agnostic software architectures suitable for education and rapid prototyping.

The absence of a flexible, low-cost, and simulator-agnostic haptic interface therefore limits the ability of researchers, educators, and students to prototype and evaluate robotic manipulation systems in a safe virtual environment. There is a clear need for a system that provides:

1. **Affordable haptic hardware** constructed from widely available, low-cost components.
2. **An open, hardware-independent communication protocol** enabling integration with arbitrary simulators or robotic platforms.
3. **A transparent haptic rendering engine** in which latency, force computation, and stability constraints are explicitly controlled.
4. **A modular architecture** that can be extended from simulation-based training to real robotic manipulators with minimal redesign.

1.3 Aims and Objectives

The aim of this project is to design, implement, and evaluate a low-cost, open-architecture two-degree-of-freedom (2-DOF) haptic interface and simulation framework capable of delivering real-time, stable force feedback for robotic manipulator training.

The specific objectives are to:

- Design and construct a 2-DOF haptic input device using low-cost BLDC actuators and magnetic rotary encoders.
- Develop embedded firmware implementing field-oriented motor control, encoder acquisition, and real-time bidirectional communication.
- Define and implement a device-agnostic packet-based communication protocol for streaming joint states and receiving force or torque commands.
- Develop a custom physics and rendering engine supporting collision detection, virtual proxy methods, and real-time force computation.
- Implement Jacobian-based mappings from end-effector interaction forces to joint-space actuator torques.
- Evaluate system performance in terms of latency, stability, and force fidelity against established haptic rendering requirements.
- Demonstrate modular integration with external simulation environments such as NVIDIA Isaac Sim or MATLAB/Simulink.

1.4 Contributions of This Work

This work presents the design and implementation of a modular real-time haptic rendering system integrating a low-cost actuated input device with a custom multithreaded simulation and visualisation framework. The system emphasises transparency of the control architecture, explicit handling of latency and stability considerations, and hardware-independent communication interfaces to support extensibility towards alternative simulators or physical robotic platforms.

2 Literature review

2.1 Haptic Interfaces for Robotics

Haptic interfaces play a central role in teleoperation, robot-assisted surgery, and operator training, where visual feedback alone is insufficient for the development of accurate motor skills and safe manipulation strategies. A large body of empirical evidence demonstrates that the inclusion of haptic feedback improves task performance, accuracy, and force regulation across a wide range of robotic manipulation tasks. A recent meta-analysis of robot-assisted surgery systems reports significant reductions in applied forces, task completion time, and error rates when haptic feedback is present, particularly for inexperienced and novice users [1].

Commercial haptic devices such as the *Geomagic Touch* are widely used in research and training environments but are typically associated with high cost and proprietary software ecosystems. Such constraints can limit adaptability, reduce transparency of the control loop, and complicate integration with custom simulation frameworks or experimental hardware platforms.

2.2 Haptic Rendering Techniques

Research in real-time haptic simulation has advanced substantially over recent decades, particularly in collision-based interaction models. Methods such as the virtual coupling and virtual proxy (or “god-object”) approach introduced by Zilles and Salisbury and later extended by Ruspini *et al.* are widely adopted due to their robustness and favourable stability properties when interacting with rigid or constrained environments [2], [3].

These approaches rely on high servo-loop update rates (typically ≥ 1 kHz), accurate end-effector pose tracking, and reliable mappings from Cartesian interaction forces to actuator torques. In robotic systems, such mappings are conventionally computed using the manipulator Jacobian, as formalised in operational-space control frameworks [4].

2.3 Stability in Haptic Systems

Ensuring stable haptic interaction places strict requirements on system latency, numerical integration, and control architecture. Foundational work by Adams and Hannaford demonstrated that unconditional stability in haptic systems can be achieved through the use of virtual coupling networks, which decouple device control from virtual environment dynamics and allow stability guarantees under passive user and environment assumptions [5].

These principles underpin many modern haptic rendering systems and highlight the importance of explicitly managing dynamic coupling, discrete-time implementation effects, and feedback loop timing in practical haptic interface design.

2.4 Low-Cost Haptic Hardware

Recent advances in low-cost hardware—including hobby-grade brushless motors, magnetic encoders, and high-performance microcontrollers—have made it increasingly feasible to construct capable actuated haptic devices using widely available components. At the same time, there is growing interest in open, modular, and reproducible research platforms that allow researchers to experiment with alternative control strategies and system architectures.

However, existing open-source software frameworks such as CHAI3D, SOFA, or Unity-based extensions frequently impose fixed control architectures, device-specific driver layers, or restrictive communication models. These constraints can complicate integration with custom hardware and limit the ability to explicitly control latency, update rates, or stability-critical elements of the haptic interaction loop.

2.5 Summary of Literature

Overall, the literature demonstrates clear benefits of haptic feedback for robotic manipulation training and establishes well-developed theoretical foundations for stable haptic rendering. Nevertheless, there remains a practical gap in the availability of integrated systems that combine low-cost actuated hardware with transparent, simulator-agnostic software architectures suitable for educational use and rapid experimental prototyping.

3 System Architecture

3.1 System Overview

The proposed system implements a bidirectional haptic interaction loop consisting of four principal layers: the haptic device hardware, embedded control firmware, host-side simulation and haptic rendering, and graphical visualisation. A conceptual overview of the signal and force flow across these layers is illustrated in Figure ??.

User motion is measured at the device joints and transmitted to the host simulation in real time. Based on the current virtual environment state, the host computes interaction forces which are mapped to joint torque commands and returned to the device actuators. This closed-loop structure enables simultaneous user influence on the simulated environment and force feedback from it.

To support stable real-time interaction, the system is explicitly partitioned into time-critical and non-time-critical subsystems. Low-level torque control and safety functions are executed on the embedded controller at high frequency, while collision detection, haptic rendering, physics simulation, and visualisation are performed on the host platform.

3.2 Design Constraints and Assumptions

The system architecture was developed to prioritise real-time stability, modularity, and reproducibility within the constraints of a low-cost laboratory prototype.

Stable haptic rendering typically requires servo update rates on the order of 1 kHz and minimal

end-to-end latency between user motion and actuator response. Accordingly, the design assumes high-frequency embedded torque control and asynchronous host-side computation.

The user is modelled as a passive element during interaction, and the virtual environment is represented using rigid-body contact dynamics. Effects such as deformable contact, high-frequency surface textures, and frictional stick–slip behaviour are considered outside the scope of this work.

Mechanical workspace, actuator torque limits, and software safety bounds are selected to enable safe desktop operation while maintaining sufficient force capability for meaningful training interactions.

3.3 Hardware Architecture

The haptic interface is based on a planar two-degree-of-freedom mechanism providing rotational motion about two orthogonal joint axes. This configuration offers a compromise between mechanical simplicity, workspace usability, and controllable force feedback while remaining representative of common robotic manipulator interactions.

Each joint is actuated by a brushless motor operating in torque control mode and instrumented with absolute angular position sensing. The mechanical structure is designed to be lightweight and back-drivable in order to reduce reflected inertia and improve transparency of the rendered interaction forces.

3.4 Embedded Control Architecture

The embedded control layer is responsible for real-time actuator torque regulation, sensor acquisition, and bidirectional communication with the host system. Torque control is implemented using field-oriented control techniques, enabling smooth force rendering across the operating range of the device.

Joint state measurements are streamed to the host at a fixed update rate, while commanded torque references are received and applied continuously. Safety mechanisms including torque limiting and communication watchdog functions are incorporated to ensure predictable behaviour under fault conditions.

3.5 Communication Architecture

A device-agnostic packet-based communication interface is used to exchange joint state and torque command data between the embedded controller and host simulation. This abstraction enables the haptic device to be integrated with different simulation environments or future physical robotic platforms without modification to the high-level control structure.

The protocol is designed to provide deterministic timing behaviour and low transmission overhead in order to support real-time haptic interaction requirements.

3.6 Host Software Architecture

Host-side software is organised as a modular multi-threaded framework comprising subsystems for world state management, physics simulation, haptic rendering, device communication, and graphical visualisation. Separation of these components allows different update rates to be maintained for computationally intensive tasks while preserving high-frequency force computation.

A structured communication mechanism is employed to coordinate data exchange between subsystems and maintain a consistent representation of the simulated environment.

3.7 Haptic Rendering Paradigm

Interaction forces are generated using a constraint-based haptic rendering approach derived from the virtual proxy (god-object) method. In this paradigm, the unconstrained device position is coupled to a proxy constrained by the geometry of the virtual environment, enabling stable rendering of unilateral contact conditions.

A virtual coupling relationship between proxy and device states provides dynamic decoupling between the physical interface and simulated environment, supporting stable interaction under discrete-time computation.

3.8 Force Transmission to the Device

Interaction forces computed in Cartesian space at the end-effector are mapped to joint-space actuator torques using the manipulator Jacobian

$$\tau = J^T(q)F. \tag{1}$$

This operational-space formulation enables physically meaningful force feedback to be rendered at the device joints while accounting for configuration-dependent kinematic relationships.

3.9 Visualisation Layer

A graphical visualisation subsystem provides real-time feedback of the simulated environment and device interaction state. Visual rendering is intentionally decoupled from the high-frequency haptic computation loop to prevent graphical latency from degrading interaction stability.

Diagnostic overlays are incorporated to support inspection of contact conditions and system behaviour during development and testing.

4 System Implementation

This section describes the practical realisation of the proposed haptic interface system, including the mechanical construction of the device, embedded firmware architecture, communication protocol, and host-side simulation and haptic rendering software.

The implementation was designed to balance low-cost hardware selection with sufficient dynamic performance to enable stable real-time bidirectional haptic interaction.

4.1 Mechanical Design of the Haptic Device

The haptic device was realised as a planar two degree-of-freedom serial mechanism providing rotational motion about two orthogonal revolute joints. This configuration was selected as it enables intuitive user interaction, allowing the end-effector to be freely positioned within a planar workspace.

A two degree-of-freedom architecture was chosen in preference to higher degree-of-freedom alternatives in order to reduce mechanical complexity and minimise the number of required actuators. This was an important design consideration to maintain the overall system within the project budget while still demonstrating meaningful bidirectional haptic interaction capabilities.

The structural components of the device were predominantly fabricated using additive manufacturing techniques. This enabled rapid iteration of mechanical designs and reduced moving mass, improving the perceived backdrivability of the interface.

The mechanism consists of two rigid links. The link geometries and overall workspace are illustrated in Figure **X**.

The link lengths were selected to provide a sufficient reachable workspace at the end-effector while ensuring that adequate torque could be delivered by the selected motors throughout the range of motion.

Actuation is provided by brushless gimbal motors (T-Motor GB2208). Each motor is mounted offset from the corresponding joint axis and torque is transmitted via a timing belt reduction stage providing an approximate transmission ratio of $3{:}1$.

Adjustable belt tensioning mechanisms were incorporated in order to minimise backlash and compensate for belt compliance.

Each joint rotates about an 8 mm diameter steel shaft supported by rolling element bearings to reduce friction and radial play. The corresponding link is attached to the shaft via a friction fit interface within the printed component, supplemented by a radial set screw to prevent slip under load.

4.2 Embedded Firmware

The embedded control system is implemented using an STM32 Nucleo-G401RE development board.

The microcontroller interfaces with two magnetic rotary encoders (AS5600) which are used to measure joint angular position.

Each encoder is mounted coaxially with the joint shaft, with a diametrically magnetised magnet attached to the shaft using a custom adapter. Angular position measurements are acquired via an SPI communication interface.

Encoder data are polled at a fixed sampling rate.

The measured joint angles are packaged into device state packets and transmitted to the host computer via a USB virtual COM interface.

The microcontroller additionally communicates with two brushless motor driver modules (BG-3421-ESC1). Torque commands received from the host system are forwarded to the motor drivers via a UART communication interface.

A communication watchdog mechanism is implemented such that if valid torque commands are not received within a predefined timeout interval, the commanded joint torques are automatically set to zero to ensure safe device behaviour.

Each motor driver executes a field-oriented control (FOC) algorithm in torque control mode. Rotor electrical angle feedback for the FOC algorithm is obtained using integrated hall-effect sensors operating in a quadrature encoder configuration.

4.3 Communication Protocol

Bidirectional communication between the host computer and the embedded controller is implemented using fixed-format binary data packets. Two primary packet types are defined: device state packets transmitted from the microcontroller to the host, and torque command packets transmitted from the host to the microcontroller.

Each packet contains a unique header identifier used for packet synchronisation, followed by joint angle or torque data and a checksum field used for basic transmission integrity verification.

Upon reception of incoming data, packet headers are validated before the payload is processed.

4.4 Host Software Architecture

The host-side software is implemented as a multithreaded C++ application composed of several modular subsystems responsible for world state management, physics simulation, haptic rendering, device communication, and graphical rendering.

4.4.1 Inter-Thread Communication Architecture

Communication between software subsystems executing on different threads is implemented using a structured message-passing architecture. This design avoids shared mutable state wherever

possible and instead enforces controlled data exchange through explicitly defined communication channels.

A central *MessageBus* object provides named access to communication channels. Two primary channel types are implemented: queue-based command channels and snapshot-based state channels.

Queue channels are implemented using a thread-safe FIFO structure protected by a mutex. These channels are primarily used for transmitting discrete event-driven messages such as world modification commands, force application requests, or tool pose updates. Messages are published by producer threads and consumed non-blockingly by the receiving subsystem. In certain subsystems, such as the physics engine, multiple queued messages may be drained and processed in batches during a single simulation step.

Snapshot channels are used for transmitting continuously updated state information between threads in a low-latency manner. These channels implement a double-buffered data structure combined with atomic index switching to allow single-producer, multi-consumer access without requiring mutex locking during read operations. A version counter is maintained to allow consumers to detect whether a new state update has been published since the previous read.

This communication architecture enables deterministic separation between the rendering loop, physics simulation loop, haptic rendering loop, and device communication loop. By distinguishing between event-driven queue channels and continuously refreshed snapshot channels, the system reduces contention between threads while maintaining timely propagation of critical state updates.

4.4.2 World State Management

The *WorldManager* subsystem acts as the authoritative owner of the virtual environment state. It is responsible for creating, modifying, and deleting world objects, and provides a structured interface through which other subsystems issue high-level commands affecting the simulation.

Each object in the virtual world is represented by a data structure containing geometric identifiers, pose information (position, orientation, and scale), visual attributes, physical properties, and semantic role labels.

Geometric shape definitions are generated via a *GeometryFactory*, where each geometry entry represents a reusable shape description rather than a specific object instance. These entries may in-

clude signed distance field (SDF) representations used for haptic collision queries, along with rendering mesh handles and optional physics shape descriptors.

A geometry database provides indexed storage and lookup functionality for these geometry entries.

4.4.3 Rendering Subsystem

Visualisation of the simulated environment is performed by a dedicated rendering subsystem implemented using OpenGL. The renderer executes on its own thread and is responsible for displaying the current state of the virtual world, visualising haptic interaction states, and providing an interactive user interface for modifying simulation parameters at runtime.

At each render iteration, the renderer first updates the camera aspect ratio using the current framebuffer dimensions and then reads the most recent world snapshot from a snapshot communication channel. This snapshot provides a read-only view of the current simulation state, including the pose, colour, geometry identifier, role, and physical properties of each object.

For each object in the world snapshot, the renderer performs a geometry lookup through the geometry database. Each geometry entry stores a lightweight render mesh handle, which is used to retrieve the corresponding GPU-resident mesh from the render mesh registry. This separation allows simulation geometry definitions to remain independent from the underlying rendering resources.

A model matrix is constructed from the object pose using its position, quaternion orientation, and uniform scale. The object is then rendered using a lightweight *Renderable* abstraction, which combines the mesh resource, shader, object colour, and rendering options before uploading the model, view, and projection transforms to the shader and issuing the final draw call.

GPU mesh resources are represented by *MeshGPU* objects, which manage the required OpenGL vertex array, vertex buffer, and element buffer objects. In the current implementation, primitive meshes such as planes, spheres, and cubes are generated once, uploaded to GPU memory, and reused across multiple scene objects via the render mesh registry.

In addition to rendering world objects, the subsystem also visualises haptic state information received from the haptic engine. The latest haptic snapshot is consumed from a queue channel and used to render overlay markers representing the current device pose and proxy pose. This provides

a useful visual indication of the relationship between the unconstrained device position and the constrained proxy position during contact interactions.

The rendering subsystem also provides interactive camera navigation. Keyboard and mouse inputs are processed through a viewport controller to allow the user to move around the virtual environment and inspect the scene from different viewpoints. Camera parameters such as field of view, near plane, far plane, yaw, and pitch are maintained as part of the renderer state.

An ImGui-based graphical user interface is layered on top of the rendered scene to support live inspection and modification of simulation parameters. Through this interface the user can select objects, adjust object pose and colour, modify physical properties such as density, damping, friction, and restitution, and create new scene objects. These edits are not applied directly to the simulation state. Instead, the UI publishes structured world modification commands to the world manager via the message bus, preserving the separation between rendering and authoritative world state ownership.

This architecture ensures that rendering remains decoupled from both the physics and haptic subsystems while still providing a responsive visual and interactive front-end for simulation control and debugging.

4.4.4 Haptic Rendering Engine

The haptic rendering engine is responsible for computing interaction forces between the user-controlled device and virtual objects in the simulated environment. The engine operates in a dedicated high-frequency loop and exchanges information with other subsystems using the message bus communication architecture described previously.

Real-Time Haptic Interaction Loop At each iteration of the haptic update cycle, the engine first retrieves the most recent world state snapshot using a snapshot channel. This provides the latest positions, orientations, and physical properties of all simulated objects.

The engine then drains any pending tool state messages received from the device communication subsystem. These messages contain the most recent estimate of the device end-effector pose and velocity expressed in the world coordinate frame.

Using this information, the engine performs a contact search across all relevant virtual objects. Objects designated as tool or proxy representations are excluded from this process. For each remaining object, a signed distance query is performed to determine the closest penetration condition between the tool point and the object surface.

If penetration is detected, a proxy position representing the constrained end-effector location on the object surface is computed. If no contact is present, the proxy position coincides with the current tool position.

Based on the resulting proxy and tool states, an interaction force is computed and limited to a predefined maximum magnitude. When contact occurs, an equal and opposite wrench command is generated and published to the physics simulation and device interface subsystems. A haptic state snapshot containing the current proxy pose, device pose, and rendered force is also published for use by visualisation or diagnostic components.

Proxy-Based Contact Rendering Contact handling is implemented using a proxy-based rendering approach. Each virtual object may provide a signed distance field (SDF) representation of its surface geometry. The tool position expressed in the world frame is first transformed into the local coordinate frame of each object under consideration.

A signed distance value ϕ and corresponding surface gradient are then obtained from the SDF. A negative signed distance indicates penetration of the tool point into the object volume. In this case, the tool position is projected onto the closest point on the surface along the direction of the SDF gradient. This projection is performed in the local object frame and subsequently transformed back into world coordinates to define the proxy position.

This mechanism allows arbitrary implicit or mesh-derived geometries to be rendered haptically using a unified contact model.

Virtual Coupling Force Model Interaction forces are generated using a virtual coupling model based on a linear spring–damper relationship between the proxy position and the actual device position. The rendered Cartesian force is given by

$$\mathbf{F} = K\left(\mathbf{x}_p - \mathbf{x}_t\right) + D\left(\mathbf{v}_p - \mathbf{v}_t\right)$$

where $\mathbf{x}_p$ and $\mathbf{x}_t$ denote the proxy and tool positions respectively, and $\mathbf{v}_p$ and $\mathbf{v}_t$ represent the corresponding velocities.

The proxy velocity is estimated using finite differences between successive proxy poses, while the tool velocity is obtained from the incoming device state message. In the present implementation the stiffness parameter is set to $K = 2000\,\mathrm{N\,m^{-1}}$ and an equivalent virtual mass parameter of $M = 0.2\,\mathrm{kg}$ is used to compute the damping coefficient according to

$$D = 0.7 \cdot 2\sqrt{KM}.$$

To ensure safe and stable interaction behaviour, the magnitude of the rendered force is limited to a maximum allowable value of $15\,\mathrm{N}$.

Coupling to the Physics Simulation When contact with a virtual object is detected, the haptic engine generates a wrench command representing the interaction force applied at the contact point. This wrench is published to the physics simulation subsystem, which applies the equal and opposite force to the corresponding dynamic actor during the next physics update step.

This bidirectional coupling ensures that user interaction influences the motion of simulated objects while simultaneously providing force feedback to the user. In addition, the haptic engine publishes a structured snapshot message containing the current device pose, proxy pose, and rendered force, enabling other subsystems such as rendering or logging modules to access the current interaction state.

4.4.5 Physics Simulation Integration

Physical behaviour of virtual objects is simulated using the NVIDIA PhysX rigid body dynamics engine. The physics engine operates as a separate subsystem that advances the dynamic state of objects based on applied interaction forces and predefined physical parameters.

At each external simulation update, the physics engine first checks whether the topology or physical properties of the virtual world have been modified. If changes are detected, all rigid actors are rebuilt from the current authoritative world state representation. This ensures consistency between the physics scene and the higher-level world model.

Interaction forces generated by the haptic rendering engine are communicated to the physics engine via a queue-based message channel. These messages contain a wrench specification including the target object identifier, applied force vector, point of application in world coordinates, and an interaction duration. In the present implementation, forces are converted into impulses using the relation

$$\mathbf{J} = \mathbf{F}\,\Delta t$$

and applied to the corresponding rigid body using PhysX impulse-based force application methods.

The physics engine advances the simulation using a fixed internal timestep. An accumulator-based integration strategy is employed, whereby incoming variable-duration external timesteps are accumulated until sufficient time has elapsed to perform one or more fixed simulation steps. This approach improves numerical stability and ensures deterministic solver behaviour.

Dynamic rigid bodies are configured with linear and angular damping parameters obtained from the world model. Continuous collision detection (CCD) is enabled to improve robustness under fast contact interactions. Solver iteration counts are explicitly configured to balance computational cost against constraint resolution accuracy.

Following each simulation update, the resulting poses of dynamic actors are written back into the authoritative world state maintained by the world management subsystem. This write-back mechanism ensures that subsequent rendering, collision queries, and haptic force computations operate on a consistent and up-to-date representation of the simulated environment.

4.4.6 jacobians maybe

5 Experimental Methodology

5.1 Communication Performance Tests

5.1.1 Packet Integrity and Rate Tests

5.1.2 Round-Trip Latency Test

5.1.3 End-to-End Loop Latency Test

5.2 Simulation Validation

5.2.1 Virtual Wall Test

5.2.2 Virtual Fixture Test

5.3 Safety Validation

5.3.1 Torque Saturation Test

5.3.2 Rate Limiting Test

5.3.3 Watchdog Timeout Test

5.4 Hardware Validation

5.4.1 Torque Step Response

5.4.2 Stable Contact Test

6 Results

6.1 Communication Performance

6.2 Haptic Model Validation

Nov. 2023, ISSN: 2045-2322. DOI: 10.1038/s41598-023-46641-8 [Online]. Available: https://doi.org/10.1038/s41598-023-46641-8 (cited on p. 8).

[2] C. Zilles and J. Salisbury, "A constraint-based god-object method for haptic display," in *Proceedings 1995 IEEE/RSJ International Conference on Intelligent Robots and Systems. Human Robot Interaction and Cooperative Robots*, vol. 3, 1995, 146–151 vol.3. DOI: 10.1109/IROS.1995.525876 (cited on p. 8).

[3] D. C. Ruspini, K. Kolarov, and O. Khatib, "The haptic display of complex graphical environments," in *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH '97, USA: ACM Press/Addison-Wesley Publishing Co., 1997, pp. 345–352, ISBN: 0897918967. DOI: 10.1145/258734.258878 [Online]. Available: https://doi.org/10.1145/258734.258878 (cited on p. 8).

[4] O. Khatib, "A unified approach for motion and force control of robot manipulators: The operational space formulation," *IEEE Journal on Robotics and Automation*, vol. 3, no. 1, pp. 43–53, 1987. DOI: 10.1109/JRA.1987.1087068 (cited on p. 9).

[5] R. Adams and B. Hannaford, "Stable haptic interaction with virtual environments," *IEEE Transactions on Robotics and Automation*, vol. 15, no. 3, pp. 465–474, 1999. DOI: 10.1109/70.768179 (cited on p. 9).

Appendices

A Project outline

Project outline as submitted at the start of the project is a required appendix. Put here.

B Risk assessment

Risk assessment is a required appendix. Put here.